

# MINCOM Appraisal Platform

Deployment Runbook

Generated from docs/deployment.md

# MINCOM Appraisal Platform — Deployment Runbook

---

This document is the authoritative step-by-step guide for deploying the MINCOM Employee Performance Appraisal Platform to a fresh DigitalOcean Droplet. Follow every numbered step in order. All commands use absolute paths.

---

## 1. Prerequisites

---

| Requirement          | Detail                                                                                                                                                                                                                        |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DigitalOcean account | With billing enabled and a project created                                                                                                                                                                                    |
| Domain name          | DNS A record pointing to the Droplet IP (see Step 2)                                                                                                                                                                          |
| SSH key pair         | Public key uploaded to DigitalOcean; private key on your local machine                                                                                                                                                        |
| GitHub PAT           | Personal Access Token with <code>write:packages</code> scope — used by the <b>Droplet</b> to pull images from GHCR ( <code>GHCR_TOKEN</code> , Step 7); CI pushes them using GitHub's own built-in token, no PAT needed there |
| Local tools          | <code>ssh</code> , <code>openssl</code> , <code>docker</code> (for local testing only)                                                                                                                                        |

---

## 2. Droplet Provisioning

---

1. Log in to the DigitalOcean Control Panel at `https://cloud.digitalocean.com` .
2. Create a new Droplet: - **Image:** Ubuntu 22.04 LTS (x64) - **Size:** 4 GB RAM / 2 vCPUs minimum (s-2vcpu-4gb or larger) - **Region:** Choose the region closest to your users - **VPC Network:** Create or select a private VPC network - **Authentication:** SSH key — select the key uploaded to your account - **Hostname:** `mincom-appraisal-prod`
3. Note the assigned IPv4 address. Add a DNS A record:  
`appraisal.<your-domain.com> IN A <DROPLET_IPv4>`
4. Set the following GitHub Actions secrets in your repository under **Settings** → **Secrets and variables** → **Actions**:

| Secret name        | Value                                                         |
|--------------------|---------------------------------------------------------------|
| DO_DROPLET_IP      | The Droplet's IPv4 address                                    |
| DO_SSH_USER        | <code>root</code> (initial) or <code>mincom</code> (hardened) |
| DO_SSH_PRIVATE_KEY | Contents of your PEM private key                              |
| GHCR_TOKEN         | GitHub PAT with <code>write:packages</code> scope             |

### 3. Server Setup

SSH into the Droplet:

```
ssh root@<DROPLET_IPv4>
```

1. Update the package index and install Docker Engine:

```
bash apt-get update apt-get install -y ca-certificates curl gnupg lsb-release install -m
0755 -d /etc/apt/keyrings curl -fsSL https://download.docker.com/linux/ubuntu/gpg \ | gpg --
dearmor -o /etc/apt/keyrings/docker.gpg chmod a+r /etc/apt/keyrings/docker.gpg echo "deb
[arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \ >
/etc/apt/sources.list.d/docker.list apt-get update apt-get install -y docker-ce docker-ce-
cli containerd.io docker-buildx-plugin docker-compose-plugin
```

2. Verify Docker is running:

```
bash docker version docker compose version
```

3. Create the application directory and system user:

```
bash useradd --system --no-create-home --shell /bin/false --uid 1001 mincom mkdir -p
/opt/mincom chown root:root /opt/mincom chmod 755 /opt/mincom
```

4. Add your SSH public key to `root`'s authorized keys (it should already be there from Droplet provisioning).  
For a hardened setup, add it for the `mincom` user:

```
bash mkdir -p /home/mincom/.ssh cp /root/.ssh/authorized_keys
/home/mincom/.ssh/authorized_keys chown -R mincom:mincom /home/mincom/.ssh chmod 700
/home/mincom/.ssh chmod 600 /home/mincom/.ssh/authorized_keys
```

### 4. Clone Repository

```
git clone https://github.com/<ORG>/mincom-appraisal.git /opt/mincom
cd /opt/mincom
```

## 5. Environment Configuration

---

1. Copy the real template to `.env.prod.local` (NOT `.env` — that name is only for local dev; `docker-compose.prod.yml` 's `php / db / Messenger-worker` services all load `.env.prod.local` explicitly via `env_file:`):

```
bash cp /opt/mincom/.env.prod.local.dist /opt/mincom/.env.prod.local chmod 600  
/opt/mincom/.env.prod.local
```

2. Edit `/opt/mincom/.env.prod.local` and fill in every blank value — `.env.prod.local.dist` itself is the authoritative, fully-commented list (including the exact `openssl rand ...` command for each secret); this table only calls out the ones that trip people up:

| Variable                               | Value                                                                                                                                                                                                                                                                          |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APP_SECRET                             | Output of <code>openssl rand -hex 32</code>                                                                                                                                                                                                                                    |
| MYSQL_DATABASE /<br>MYSQL_USER         | Pre-filled ( <code>mincom_appraisal_symfony</code> / <code>mincom_symfony</code> ) — leave as-is unless you have a reason to change them, but if you do, update <code>DATABASE_URL</code> to match (Symfony doesn't cross-reference the two)                                   |
| MYSQL_PASSWORD                         | Output of <code>openssl rand -hex 24</code> — must match the password embedded in <code>DATABASE_URL</code> below                                                                                                                                                              |
| MYSQL_ROOT_PASSWORD                    | Output of <code>openssl rand -hex 24</code> — only needed for manual root access inside the <code>db</code> container                                                                                                                                                          |
| DATABASE_URL                           | Replace <code>REPLACE_WITH_MYSQL_PASSWORD</code> with the real <code>MYSQL_PASSWORD</code> value above                                                                                                                                                                         |
| JWT_PASSPHRASE                         | Output of <code>openssl rand -hex 32</code> — the keypair itself is generated automatically on first boot (see Step 6)                                                                                                                                                         |
| TURNSTILE_SECRET_KEY                   | Your Cloudflare Turnstile secret — must be non-empty in prod, or CAPTCHA verification is silently skipped entirely                                                                                                                                                             |
| REDIS_URL /<br>MESSENGER_TRANSPORT_DSN | Pre-filled, plain <code>redis://redis:6379</code> (no TLS, no password) pointing at this file's own bundled <code>redis</code> service — only change these if you deleted that service in favor of an external Redis                                                           |
| FRONTEND_URL                           | Replace <code>REPLACE_WITH_PRODUCTION_DOMAIN</code> with the real domain — <b>also update the same domain in <code>docker/nginx/default.prod.conf</code>'s <code>ssl_certificate</code> / <code>ssl_certificate_key</code> paths before building the nginx image (Step 10)</b> |
| MAILER_DSN                             | Replace both <code>REPLACE_WITH_POSTMARK_TOKEN</code> placeholders with your Postmark server token (used as both SMTP username and password; percent-encode any <code>@</code> in it as <code>%40</code> )                                                                     |
| MAILER_FROM_ADDRESS                    | Replace <code>REPLACE_WITH_PRODUCTION_DOMAIN</code> with a verified Postmark Sender Signature address                                                                                                                                                                          |
| AUDIT_HMAC_KEY                         | Output of <code>openssl rand -hex 32</code> (must differ from <code>FIELD_ENCRYPTION_KEY</code> and <code>APP_SECRET</code> )                                                                                                                                                  |
| FIELD_ENCRYPTION_KEY                   | Output of <code>openssl rand -hex 32</code> — see the "FIELD_ENCRYPTION_KEY — Critical Warning" section below before you generate this                                                                                                                                         |
| SENTRY_DSN                             | Your Sentry project DSN — leave blank to disable (not an error)                                                                                                                                                                                                                |

`App\Service\ProductionConfigValidator` refuses to boot under `APP_ENV=prod` if `APP_SECRET` / `TURNSTILE_SECRET_KEY` are still blank, if `AUDIT_HMAC_KEY` / `FIELD_ENCRYPTION_KEY` / `JWT_PASSPHRASE` still match the placeholder values committed in the repo's own (dev-only) `.env`, or if `DATABASE_URL` still contains `REPLACE_WITH_MYSQL_PASSWORD` — a misconfigured deploy fails loudly on first request/command instead of silently running insecure.

There is no Symfony equivalent of Django's `DEBUG`, `ALLOWED_HOSTS`, or `DJANGO_SETTINGS_MODULE` — `APP_ENV=prod` (the first line of `.env.prod.local.dist`) covers environment selection, and Symfony trusts all

hosts by default behind the bundled nginx.

`APP_VERSION` is deliberately NOT one of these — it's a *shell* variable you export before running `docker compose`, used only to pick which image tag to pull/build (see Step 8), not something the PHP application itself reads from `.env.prod.local`.

---

## 6. JWT Signing Keys

---

The application signs authentication tokens with an RS256 key pair (lexik/jwt-authentication-bundle). Unlike the old Django setup, there is no manual key generation step and no `secrets/jwt_private.pem` / `secrets/jwt_public.pem` files to create — `entrypoint.sh` generates the keypair automatically on first container boot (idempotent; it skips generation if a keypair already exists) and persists it in the named Docker volume `jwt_keys`, mounted at `/app/config/jwt` inside the `php` service (and each Messenger worker, which sets `SKIP_INIT=true` so only the `php` service's boot actually runs migrations/keypair-generation — see `docker-compose.prod.yml`).

All you need to provide is the passphrase that protects the keypair:

```
# In /opt/mincom/.env.prod.local
JWT_PASSPHRASE=<output of `openssl rand -hex 32`>
```

**The `jwt_keys` volume must persist across container recreation/redeploys.** If it's ever removed, every restart mints a new keypair, invalidating every outstanding access/refresh token, and (in a multi-replica setup) each replica would sign with a different key. Do not run `docker compose down -v` on this volume in production. Back up `JWT_PASSPHRASE` itself to a secure secrets manager (e.g., DigitalOcean Spaces with SSE, HashiCorp Vault, or 1Password Secrets Automation) — losing it makes the persisted keypair undecryptable.

---

## 7. GHCR Authentication

---

CI ( `.github/workflows/deploy.yml` ) pushes images using GitHub's own built-in `GITHUB_TOKEN` — no PAT needed on that side. `GHCR_TOKEN` is for the **Droplet**, so it can pull those images: authenticate the Docker daemon there once (credentials persist in `~/.docker/config.json` — `docker login` doesn't need repeating on every deploy):

```
echo <GHCR_TOKEN> | docker login ghcr.io -u <GITHUB_USERNAME> --password-stdin
```

This writes credentials to `/root/.docker/config.json` (or `~mincom/.docker/config.json` for the service user). Verify with:

```
docker pull ghcr.io/<GITHUB_ORG>/mincom-appraisal-symfony-backend:latest
```

(Both images this repo builds are named `mincom-appraisal-symfony-backend` and `mincom-appraisal-nginx`, under `ghcr.io/<GITHUB_ORG>/` — see `docker-compose.prod.yml` and `.github/workflows/deploy.yml`.)

---

## 8. First Start

---

**Before the first start**, edit `docker/nginx/default.prod.conf` and replace

`REPLACE_WITH_PRODUCTION_DOMAIN` in the `ssl_certificate` / `ssl_certificate_key` paths with the real domain (same value as `FRONTEND_URL` in `.env.prod.local`) — nginx can't read environment variables for those directives, so this is a one-time manual edit, committed to the repo before CI builds the image. You'll also need Let's Encrypt certificates to already exist at that path before nginx can start successfully — see Step 10, which has to run once with nginx stopped before this step can fully succeed.

Doctrine migrations run automatically via the container entrypoint (`entrypoint.sh` runs `php bin/console doctrine:migrations:migrate --no-interaction` before handing off to the container's CMD) — no manual migration step is required.

`docker-compose.prod.yml` is a standalone stack, not an override file for `docker-compose.yml` (that file is for local dev only) — always pass just the one `-f`:

```
cd /opt/mincom
export APP_VERSION=<short-sha-or-tag> # defaults to "latest" if unset
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Wait for all 6 services to reach a healthy state:

```
docker compose -f docker-compose.prod.yml ps
```

Expected output shows `Up` or `running` for: `nginx`, `php`, `messenger-consume-scheduler`, `messenger-consume-notifications`, `db`, `redis`. (`nginx` talks directly to `php` over FastCGI — there's no separate reverse-proxy layer between them; `messenger-consume-scheduler` and `messenger-consume-notifications` replace Celery/Celery Beat, running `php bin/console messenger:consume async scheduler_main` and `php bin/console messenger:consume notification_email` respectively.)

---

## 9. Seed Data

---

No manual step is required here. Reference data (BSC Perspectives, Competencies, Score Descriptors) ships as data migrations under `migrations/` and is applied automatically as part of Step 8's

`doctrine:migrations:migrate` run, alongside the initial `BOOTSTRAP_ADMIN_*` / `BOOTSTRAP_SUPERUSER_*` account bootstrap (also run automatically by the entrypoint — see `docs/ops/django-admin.md` for details on those two commands).

`php bin/console app:seed-dev-data` exists but is for local development/demo data only (sample employees, an in-progress appraisal cycle, etc.) — do **not** run it against production.

---

## 10. SSL Certificate Provisioning

---

### Install Certbot

No `python3-certbot-nginx` plugin — nginx runs **inside a container** here, with `default.prod.conf` baked into the image at build time, so certbot's `--nginx` plugin has nothing on the host it can autoconfigure or reload. Plain `certbot` is enough; issuance and renewal both just obtain the certificate files and this container picks them up via a read-only bind mount ( `docker-compose.prod.yml` 's `/etc/letsencrypt:/etc/letsencrypt:ro` ), same as the standard "external nginx" Certbot deployment pattern, not the "certbot manages nginx for you" one.

```
sudo apt install -y certbot
```

### Verify DNS

Confirm the domain resolves to the Droplet's IP before requesting a certificate:

```
dig +short <your-domain.com>
# Must return the Droplet's IPv4 address
```

### Issue the Certificate

`--standalone` needs port 80 free to complete the HTTP-01 challenge — on a brand-new Droplet nothing is listening yet, so this is naturally the first time you'll touch SSL; on a Droplet where the stack is already up, stop the `nginx` container first (it's the only thing publishing port 80):

```
cd /opt/minicom
docker compose -f docker-compose.prod.yml stop nginx # skip on a fresh Droplet — nothing
to stop yet
sudo certbot certonly --standalone -d <your-domain.com> -d www.<your-domain.com>
```

Replace `<your-domain.com>` with the production domain — the same value you used for `REPLACE_WITH_PRODUCTION_DOMAIN` in `docker/nginx/default.prod.conf` and `FRONTEND_URL` in `.env.prod.local` (Step 8).

### Fix Read Permissions for Rootless Nginx Container

The Nginx container runs as UID 1001 (non-root). Certbot sets `/etc/letsencrypt/live/` to `0700` (root-only). Grant world-read/execute access after issuance:

```
sudo chmod -R o+rx /etc/letsencrypt/live/ /etc/letsencrypt/archive/
```

Run this command again after every certificate renewal.



## Start (or Restart) Nginx to Load the Certificate

```
docker compose -f /opt/mincom/docker-compose.prod.yml up -d nginx
```

## Certificate Paths Used by Nginx

```
/etc/letsencrypt/live/<your-domain.com>/fullchain.pem  
/etc/letsencrypt/live/<your-domain.com>/privkey.pem
```

These paths are mounted read-only into the Nginx container via:

```
- /etc/letsencrypt:/etc/letsencrypt:ro
```

## Automatic Renewal

Certbot installs a systemd timer or cron job at `/etc/cron.d/certbot` that runs `certbot renew` twice daily — but plain `certbot renew` would fail here whenever a renewal is actually due, since `--standalone` needs port 80 free and the `nginx` container is normally sitting on it. Register hooks so renewal stops/restarts just that one container around the actual challenge, and re-applies the permission fix from above (a successful renewal writes fresh files under `/etc/letsencrypt/archive/`, which need the same `o+rx` fix every time):

```
sudo mkdir -p /etc/letsencrypt/renewal-hooks/pre /etc/letsencrypt/renewal-hooks/post  
  
sudo tee /etc/letsencrypt/renewal-hooks/pre/stop-nginx.sh > /dev/null <<'EOF'  
#!/bin/sh  
docker compose -f /opt/mincom/docker-compose.prod.yml stop nginx  
EOF  
  
sudo tee /etc/letsencrypt/renewal-hooks/post/start-nginx.sh > /dev/null <<'EOF'  
#!/bin/sh  
chmod -R o+rx /etc/letsencrypt/live/ /etc/letsencrypt/archive/  
docker compose -f /opt/mincom/docker-compose.prod.yml up -d nginx  
EOF  
  
sudo chmod +x /etc/letsencrypt/renewal-hooks/pre/stop-nginx.sh \  
               /etc/letsencrypt/renewal-hooks/post/start-nginx.sh
```

Certbot runs every script under `renewal-hooks/pre` and `renewal-hooks/post` automatically on every `certbot renew` — no further manual action is required.

## Renewal Test

```
sudo certbot renew --dry-run
```

This command must exit 0 (and briefly stop/restart the `nginx` container, per the hooks above — that's expected). Run it after the initial setup to confirm the renewal configuration is working correctly.

## 11. Smoke Test

---

1. Check all 6 services are running:

```
bash docker compose -f /opt/mincom/docker-compose.prod.yml ps
```

Expected: `nginx`, `php`, `messenger-consume-scheduler`, `messenger-consume-notifications`, `db`, `redis` all show `Up` or `running`.

2. Verify the health endpoint responds:

```
bash curl -sf https://<your-domain.com>/api/v1/health/ | python3 -m json.tool
```

3. Test API login:

```
bash curl -X POST https://<your-domain.com>/api/v1/auth/login/ \ -H "Content-Type: application/json" \ -d '{"identifier": "<ADMIN_EMAIL>", "password": "<ADMIN_PASSWORD>"}' \ | python3 -m json.tool
```

A successful response returns `access` and `refresh` tokens.

4. Check application logs for errors:

```
bash docker compose -f /opt/mincom/docker-compose.prod.yml logs --tail=50 php
```

(`nginx` only carries its own access/error logs; application errors and Monolog output come from the `php` service.)

---

## 12. Subsequent Deployments

All deployments after the first are handled automatically by the CI/CD pipeline. On every push to `main`, GitHub Actions (`.github/workflows/deploy.yml`):

1. Builds and pushes both images to GHCR, tagged with the commit SHA and `latest` (`build-and-push` job)
2. SSHes to the Droplet, `git reset --hard origin/main` (so `docker-compose.prod.yml` itself and any config changes land too, not just the images), then `docker compose pull && docker compose up -d` using that same SHA as `APP_VERSION` (`deploy` job)

No manual intervention is required — including for a rollback: re-run the workflow from the GitHub Actions UI ("Run workflow") with the `version` input set to an older commit SHA or previously-pushed tag. To trigger a manual redeploy from the Droplet itself instead:

```
cd /opt/mincom
export APP_VERSION=<target-sha-or-tag> # defaults to "latest" if unset
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

## Backup and Restore

---

### Scheduling Backups

Add the backup script to the root crontab on the Droplet:

```
crontab -e
```

Append the following line:

```
0 2 * * * /opt/mincom/scripts/backup.sh >> /var/log/mincom_backup.log 2>&1
```

This runs the backup daily at 02:00 UTC. The script retains 30 daily dumps and deletes files older than 30 days automatically.

### Verifying a Backup

1. List backup files:

```
bash ls -lh /opt/mincom/backups/
```

2. Test the integrity of a dump file (must exit 0):

```
bash gunzip -t /opt/mincom/backups/<FILENAME>.sql.gz
```

### Restore Procedure

1. Source `.env.prod.local` first so `$MYSQL_PASSWORD`, `$MYSQL_USER`, and `$MYSQL_DATABASE` are in scope for the next two steps (`MYSQL_PWD` avoids the password showing up in `ps` output, the same concern the old Postgres `PGPASSWORD` pattern had):

```
bash source /opt/mincom/.env.prod.local
```

2. Stop application services (leave `db` and `redis` running):

```
bash docker compose -f /opt/mincom/docker-compose.prod.yml \ stop nginx php messenger-  
consume-scheduler messenger-consume-notifications
```

3. Restore the database from a dump:

```
bash gunzip -c /opt/mincom/backups/<FILENAME>.sql.gz \ | docker compose -f  
/opt/mincom/docker-compose.prod.yml \ exec -T -e MYSQL_PWD="$MYSQL_PASSWORD" db mysql -u  
"$MYSQL_USER" "$MYSQL_DATABASE"
```

4. Restart all services:

```
bash docker compose -f /opt/mincom/docker-compose.prod.yml up -d
```

5. Run the post-restore smoke test (see section below).

## FIELD\_ENCRYPTION\_KEY — Critical Warning

**WARNING:** The `FIELD_ENCRYPTION_KEY` in `.env.prod.local` encrypts all PII fields in the database. If this key is lost, all encrypted employee data is permanently unrecoverable even with a valid database backup. The key **MUST** be backed up separately from the database dump — store it in a secrets manager (e.g., DigitalOcean Secrets, HashiCorp Vault, or an offline secure location). Never store the encryption key in the same location as the database backup.

Checklist for key custody: - [ ] `FIELD_ENCRYPTION_KEY` stored in a secrets manager before go-live - [ ] At least two authorised personnel have access to the key - [ ] Key rotation procedure documented in the team runbook - [ ] Key value confirmed different from `APP_SECRET` and `AUDIT_HMAC_KEY`

## Post-Restore Smoke Test

1. Validate the schema and container configuration:

```
bash docker compose -f /opt/mincom/docker-compose.prod.yml \ exec php bin/console doctrine:schema:validate
```

Must exit 0 with no errors (confirms the restored database matches the applied Doctrine migrations).

2. Verify the health endpoint:

```
bash curl -sf https://<your-domain.com>/api/v1/health/
```

3. Attempt login with a known account to confirm PII decryption is working (this exercises the `FIELD_ENCRYPTION_KEY` path):

```
bash curl -X POST https://<your-domain.com>/api/v1/auth/login/ \ -H "Content-Type: application/json" \ -d '{"identifier": "<KNOWN_EMAIL>", "password": "<KNOWN_PASSWORD>"}'
```

A `200 OK` with token fields confirms the encryption key is correct and data is readable.